

NATS clustering: how far do three nodes actually **get you?**

Three demos in this lab were built to answer one growth question, and every claim on the slides that follow is a measured number, not a design opinion.

THE QUESTION

We start on one three-instance cluster. When must we add a second cluster, a gateway, an arbiter site or a leaf node — and what does JetStream cost us at each step?

THE EVIDENCE BEHIND THESE SLIDES

■ Demo 02 — multi-region · 6 servers, gateway, accounts

■ Demo 03 — 9 topologies · 196 rig checks, 0 failed

■ Demo 04 — JetStream + CQRS · rehydration cost

01

WHAT WAS ACTUALLY RUN

Three demos, one growth question

Each demo holds everything still except one variable. Demo 02 varies the account model on a fixed topology. Demo 03 varies the topology itself. Demo 04 drops to a single server and measures what JetStream costs when you rebuild state from the log.

DEMO 02 — MULTI-REGION

- › **6 servers**, two clusters of three, joined by one **gateway**.
- › Variable: the **account model**. One shared account, or one per region.
- › Answers: does a tenant wall hold across a gateway? What does **Replicas: 1** cost? Where does the data physically live?

DEMO 03 — TOPOLOGY AS THE VARIABLE

- › **9 runnable topologies** — five shapes and four variants — needing 6, 7 or 9 servers.
- › Every shape built from nothing by a script, measured, then torn down.
- › **196 rig checks passed, 0 failed**, plus 71 recorded observations. A check has an expected answer and passes only on an exact match.
- › T4 ↔ T5 switch: 83 procedure checks met, 18 not met, 3 procedure verdicts — all failed. Slide 14.

DEMO 04 — JETSTREAM + CQRS

- › **1 server**. No cluster, no gateway, no domain.
- › Variable: rebuild an aggregate **with** a snapshot, or **without** one.
- › Answers: what does replay cost, and how fast does that cost grow?

THE FIVE WORDS THAT KEEP GETTING CONFUSED

WORD	QUESTION IT ANSWERS	PLAIN MEANING
Region	Where does it run?	A place. Cape Town. Sydney.
Tenant	Whose is it?	A paying customer. A commercial fact, not a running thing.
Account	What may it touch?	A NATS wall. It decides what you may see and send.
Cell	What ships together?	One stamp of the stack, deployed as a unit.
JetStream domain	Which storage pool holds the log?	Which NATS storage the messages actually live in.

WHY THIS SLIDE EXISTS

These five are **independent axes**. No two may be defined as permanently equal. One V3 drawing used the word “**domain**” **26 times** on one page, carrying three unrelated meanings.

Only one is enforced by a machine: **NATS enforces the account wall**. The rest are conventions we hold ourselves. Region and cell are places; tenant and account are not.

02

FIVE WORDS USED ON EVERY SLIDE THAT FOLLOWS

Instance, cluster, gateway, leaf — and JetStream namespace

The shape below is the one we would actually deploy: two AWS regions, three availability zones each, one NATS server per zone. Three zones is why a cluster is three servers — lose a zone and two of three still holds a majority.



1 · NATS INSTANCE

One server process. It holds data and answers clients. Here, one per availability zone.

2 · CLUSTER

Three or more instances that **vote together**, inside one region. This is the unit of availability, and the only thing three servers buys you.

3 · GATEWAY

Joins two clusters and routes messages between them. It does **not** copy data — and it fuses both sides into one **meta group**: the set of servers that vote on every stream change. See slide 06.

4 · LEAF NODE

A server that borrows a hub over one outbound link. It may run its own JetStream. A leaf link is the only place **jetstream.domain** may change. See slide 10.

5 · JETSTREAM NAMESPACE

The stream and KV names **one meta group** manages. **Not the account**, which owns the subjects. A name is unique per **meta group × account** — change either one and the collision goes away.

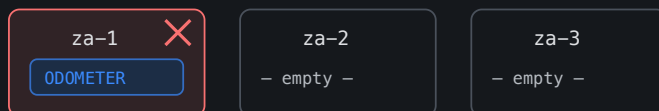
03

THE THREE-INSTANCE CLUSTER

Three servers is not three copies

A cluster is three instances because RAFT needs a majority to elect a leader, and the majority of three is two — so one instance can be lost. That protects the *cluster*. It does not protect your *stream*. Replication is a separate, per-stream choice, and its default is **one**.

REPLICAS: 1 — DEFAULT

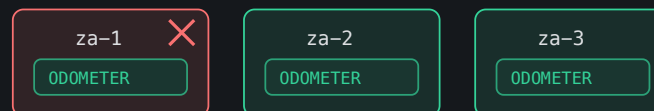


stop the one server holding it

stream is offline (10118)

DATA STUCK UNTIL THAT SERVER RETURNS

REPLICAS: 3 — ASKED FOR, PER STREAM



stop the stream-leader

new stream-leader in about one second

NOTHING LOST · EVERY WRITE NOW COSTS THREE SERVERS

THE TRAP

- › The default is **1**, and nothing warns you. A three-server cluster with an R1 stream survives nothing.
- › **Replicas never cross a region.** `--replicas 3` means three copies in **za** — not one in each region.
- › R3 is slower and uses three times the disk. Choose it **on purpose, per stream**.

WHAT `--REPLICAS 3` ACTUALLY COPIESCOPIED 3 TIMES, INSIDE **ZA**

- › **Stream messages** — the log itself.
- › **Stream config** — subjects, limits, retention.
- › **Consumer position and acks** — its *own* vote group, which inherits 3 from the stream.
- › **KV bucket values** — a KV bucket **is** a stream.
- › **Object store blobs** — also a stream.

HANDLED SOMEWHERE ELSE

- › The **fact** that the stream exists. That lives in the meta group, on every server in the meta group — not in the replica set.

NOT COPIED AT ALL

- › **Anything in **au**** . A gateway routes; it does not copy. That needs a mirror.
- › **Ephemeral consumers, by default** — on a limits-policy stream with no replica count set, an ephemeral is R1 and dies with its server. Set a replica count to replicate it; on other retention policies it takes the stream's count. [source](#)

RECOMMENDATION

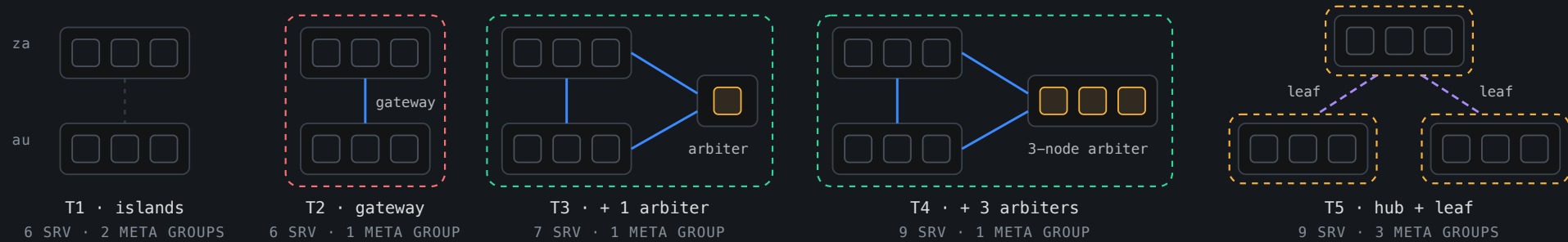
Start at three instances, one cluster. Set **Replicas: 3** explicitly on every stream and KV bucket that matters. Make it a reviewed default, not a habit.

04

FIVE WAYS TO ADD A SECOND REGION

Every shape is majority arithmetic

Demo 03 built nine topologies from nothing and measured each one. The single question that separates them is: **how many servers are in the JetStream meta group, and who still holds a majority when a region goes dark?** This slide only introduces them. The findings follow, one shape at a time.



Red ring = one shared fate. Green ring = one shared fate that still holds a majority when a region is lost. Amber rings = separate meta groups, each with its own vote.

THE FOUR VARIANTS

VARIANT	WHAT IT ASKS
B · domain per cluster	T2, plus a different <code>jetstream.domain</code> on each side. Can a domain name split a supercluster?
E · export / import	T2, plus two accounts instead of one. Can a region share data deliberately, across the account wall?
G · T6	A gateway and a hub leaf link at once. Does the leaf link change what the gateway does?
H · T5 + accounts	T5 with an account per region. Does that remove double capture?

WHAT EACH SHAPE IS

SHAPE	SERVICES	META GROUPS	WHAT IT IS
T1 islands	6	2	Two clusters, no link at all. Each region is its own NATS and its own JetStream.
T2 gateway	6	1	The same two clusters, joined by one gateway.
T3 + 1 arbiter	7	1	T2 plus a single-node third site that also votes.
T4 + 3 arbiters	9	1	T3 with a full three-node third site instead of one node.
T5 hub + leaf	9	3	A hub cluster, with a leaf cluster in each region. Each runs its own JetStream.

WHAT FOLLOWS

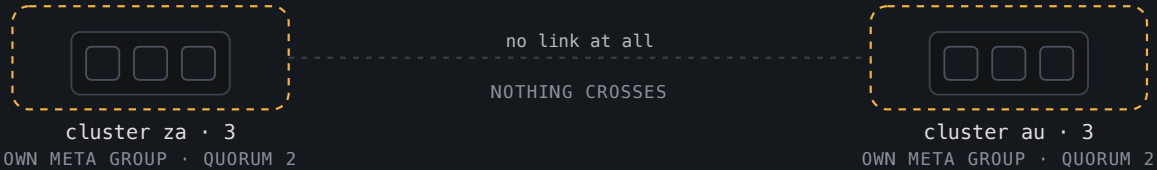
One finding set per shape: **05** T1 · **06–07** T2 · **08** B · **09** T3 and T4 · **10** T5 · **11** H · **12** E · **13** G · **14** the T4 ↔ T5 switch. Every row on 05–13 is a rig check with an expected answer that matched. Slide 14 is a procedure, not a shape, and it failed.

05

T1 · NO LINK AT ALL · 6 SERVERS · 12 CHECKS, 12 PASSED

Two islands never freeze, and never agree

The baseline. Two clusters of three, with nothing joining them. Each region runs its own JetStream and its own vote. This is the only shape where a region going dark changes nothing at all for the other one — and the only shape that shares nothing whatsoever.



WHAT IT GIVES YOU

- › **Two separate meta groups**, of three servers each. A majority of three is two, so each region tolerates losing one server. **T1a** , **T1b** , **T1c**
- › **ZA goes dark and AU does not notice**. AU keeps its meta leader, creates a brand-new 3-replica stream, and its meta group size is unchanged at 3. **T1g** , **T1h** , **T1i**
- › No WAN on any read or write. Nothing to fail between the regions, because there is nothing between them.

WHAT IT COSTS YOU

- › **Two streams can carry the same name and be unrelated**. AU asks for **ODOMETER** , gets it, and it is a second real stream. No error, because there is no shared JetStream namespace to collide in. **T1d** , **T1e**
- › **Nothing crosses**. One publish in ZA: ZA holds 1 message, AU holds 0. **T1f**
- › Every cross-region copy is an application you have to write, run and monitor yourself.

MEASURED — DEMO 03, T1

CHECK	QUESTION	EXPECTED	MEASURED
T1a	Distinct JetStream meta groups across both regions	2	2
T1f	One publish in ZA — messages in ZA / AU	1 / 0	1 / 0
T1g	ZA dark — AU still has a meta leader	yes	yes

CHOOSE T1 WHEN

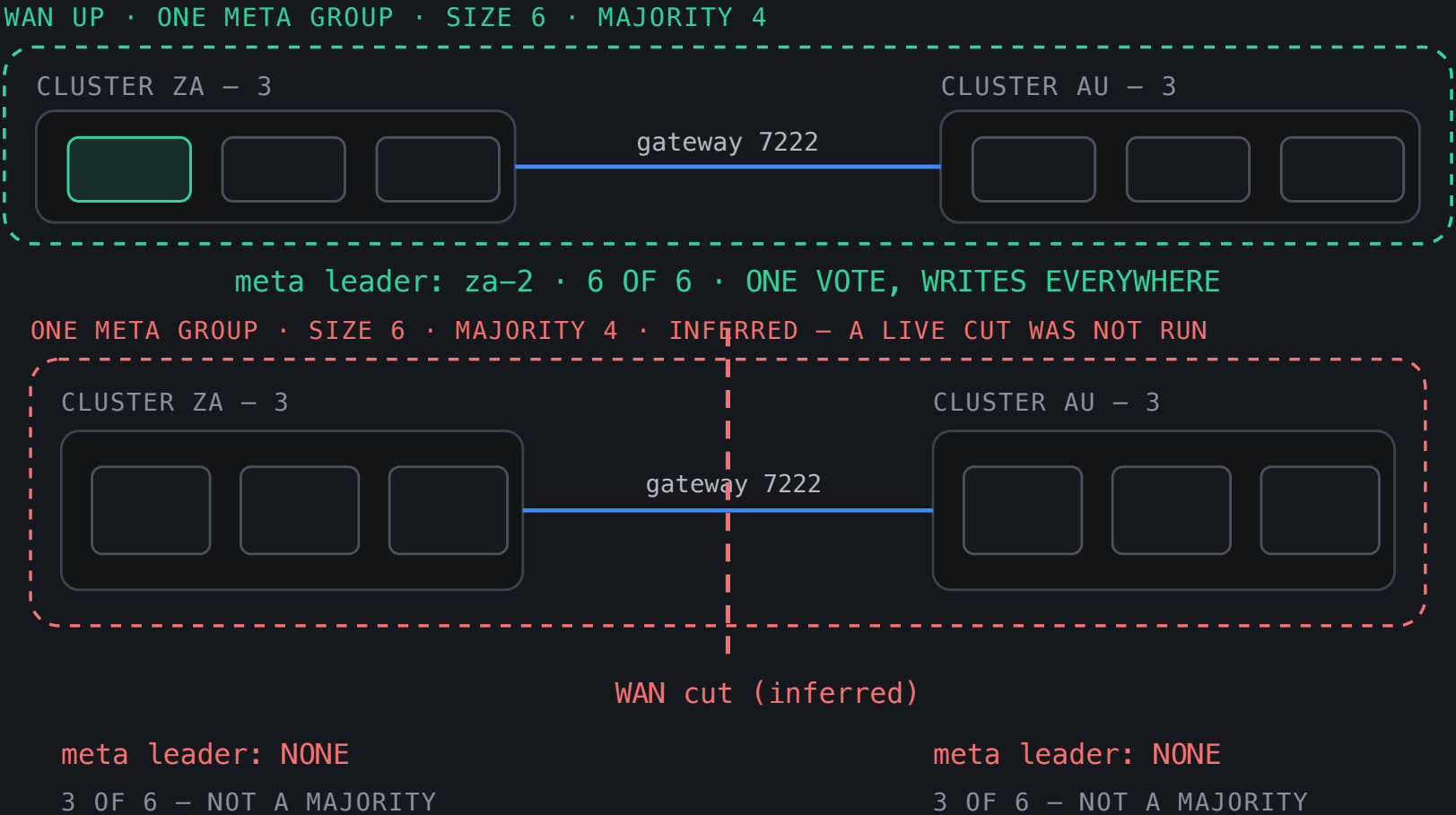
The two regions genuinely share no data, and you would rather write one explicit copier than accept a shared vote. It is the safest shape for availability and the worst for consistency — and those are the same fact.

06

THE MOST EXPENSIVE THING ON THESE SLIDES

A gateway makes one JetStream meta group, not two

A gateway does not link two meta groups — it fuses them into one supercluster with **one meta group** and **one leader election**. The account wall still holds for data. It does not hold for availability.



READ THE TWO DRAWINGS TOGETHER

Only the **link** changed — and with it, the arithmetic. Six votes become three, three is not four, and every JetStream **change** stops in both regions at once. Measured: a whole region **stopped**. A live two-sided cut is inferred.

07

DEMO 03 · T2 / A · 6 SERVERS · 26 CHECKS — ALL PASSED

One JetStream namespace, one copy, and one shared fate

The previous slide showed the arithmetic. These are the consequences you will actually meet: losing a whole region is the same sum as losing the link, names collide, data lives in exactly one place, an uneven split behaves differently — and some things carry on regardless.

IF A WHOLE REGION DIES, NOT JUST THE LINK

- › The meta leader's **location** does not matter. Lose any 3 of 6 and you are under 4. [A10](#)
- › AU still publishes into the streams **it** holds. It cannot create anything new. [A11](#) , [A12](#)
- › ZA's streams are not in AU. A gateway **routes**; it never copies. [A8](#)
- › [/jsz](#) went on naming the dead meta leader — **for up to about a minute**. [A10a](#)

ONE JETSTREAM NAMESPACE MEANS ONE NAME

- › AU asks for a stream ZA already owns → [10058 stream name already in use](#)
- › Overlapping subjects → [10065 subjects overlap](#)
- › So inside one account, two streams cannot **capture** the same subject. A mirror, a source or a repeated publish can still make a second copy.
- › **Different accounts, no collision.** [LB_ZA](#) and [LB_AU](#) may each own an [ODOMETER](#) — the wall is the **account**, not the gateway. Slide 12, [E7](#) , [E8](#)

...AND ONE COPY, IN ONE PLACE

- › AU's client keeps working, because a stream is reachable from either side of a gateway.
- › But [stream info](#) from AU reports cluster **za**. AU holds nothing.
- › AU pays the WAN on every read and goes dark for that stream when ZA is lost.

UNEVEN REGIONS BEHAVE DIFFERENTLY

- › A **3 / 3** split freezes both sides.
- › A **3 / 2** split does not — the side holding 3 of 5 keeps its leader.
- › Which side survives is arithmetic, so design the count deliberately.

WHAT A 3/3 SPLIT FREEZES

- › [stream add / rm / edit](#) → [10008 JetStream system temporarily unavailable](#)
- › Every account at the same moment. Measured with 3 JetStream accounts live: **1 meta group**, not three.

WHAT KEEPS WORKING

- › Core NATS publish and subscribe.
- › Reads from a stream **your own region holds**. [A20](#)
- › **Writes into a stream that already exists** — measured 10 → 15 messages, all acked.
- › **Not** a read of a stream the dark region held — that fails [10008](#) . [A21](#) , [A21a](#)

THE DECISION

If both regions must accept a JetStream **change** while the other is dark, a plain two-region gateway cannot do it. Add a vote outside both regions, or stop sharing a meta group.

08

THE SETTING EVERYBODY REACHES FOR FIRST

A domain may only change across a leaf link

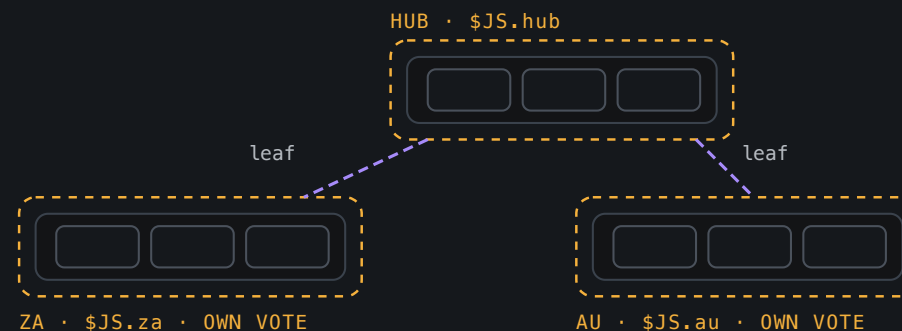
`jetstream.domain` looks like the fix for the previous slide. It is not. A **different** domain on each side of a gateway made things quietly worse; one domain across a supercluster splits nothing. Over a **leaf** link a domain per side is real: it stops JetStream extending across the link. It is not a security boundary. [NATS docs: leaf nodes](#)

A · DOMAIN OVER A GATEWAY — DOES NOT WORK



meta group size still 6 — domains did NOT split it
 one side never elects · placement → 10005 no suitable peers
 AND WHICH SIDE GOES BLIND IS A COIN TOSS — MEASURED BOTH WAYS

B · DOMAIN OVER A LEAF LINK — REAL



OVER A GATEWAY — MEASURED

- › Meta group size **still 6**. A domain name cannot split a supercluster.
- › One cluster elects, the other reports `leader: null` forever.
- › Placing a stream on the blind cluster → `10005 no suitable peers for placement`
- › The verdict in the report is blunt: **does not work**.

OVER A LEAF LINK — MEASURED

- › **3 distinct meta groups**, sizes 3 / 3 / 3. Hub, ZA and AU each own their JetStream.
- › The whole hub can be stopped and both regions still elect, still create R3 streams, still store writes.
- › What sets T5 apart is an **independent quorum per region**. T6 also survives hub loss (`G16 – G18`), but its regions still share one vote.

THE PRICE NOBODY MENTIONS

- › **Double capture is real here** on a shared account. One publish in ZA was stored **1 and 1** — twice, with no warning. Per-region accounts remove ZA / AU cross-capture, not hub capture (slide 11).
- › **Nothing replicates by itself**. Every cross-region copy is hand-written.
- › A mirror over a leaf needs `mirror.external.api` → `$JS.au.API`. Without it: **0 messages copied**, and the stream reports healthy.

THE RULE

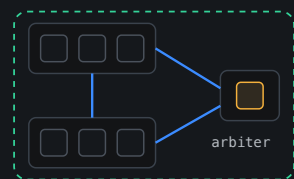
Use one domain across a supercluster, or none. A different domain per gateway-joined cluster does not split the meta group — it leaves one cluster blind while the config reports success. Change the domain only across a leaf link.

09

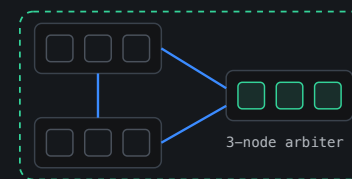
T3 · 7 SERVERS · 18 CHECKS — T4 · 9 SERVERS · 26 CHECKS — ALL PASSED

Buy a vote that sits outside both regions

T2 freezes because 3 of 6 is not a majority. Both fixes put more voters in a third site, so a surviving region plus that site still adds up. T3 adds one node, T4 adds three, and the difference is **slack**.



T3 · + 1 arbiter
7 SRV · MAJORITY 4 · ZERO SLACK



T4 · + 3 arbiters
9 SRV · MAJORITY 5 · ONE NODE OF SLACK

T3 — ONE ARBITER NODE · 7 SERVERS, MAJORITY 4

- One meta group across all three sites. [C1](#) , [C2](#)
- Either region can go dark and the survivors elect a leader** and still create a new 3-replica stream. [C3](#) — [C6](#)
- Zero slack.** 4 of 7 is exactly the majority. One more node after the region loss and it freezes. [C12](#)

T4 — A THREE-NODE ARBITER SITE · 9 SERVERS, MAJORITY 5

- Still one meta group, size 9. [F1](#) , [F2](#)
- A region dark leaves 6 of 9 — a leader is elected and writes continue. [F3](#) , [F4](#)
- One node of slack.** A region *plus* one arbiter node leaves 5 of 9, exactly the majority, and it still works. At 4 of 9 the leader is NONE. [F5](#) , [F6](#) , [F7](#) , [F11](#)
- Even with no **meta** majority, publishing into an **existing** stream still works — provided that stream keeps its own quorum. Only changes stop. [F8](#) , [F9](#) , [F21](#)

THE ARBITER TRAP — IT WILL QUIETLY HOST YOUR DATA

- An explicit `--cluster arb` R1 stream **lands on the arbiter**, and a client dialling the arbiter's own port creates streams there too. [C8](#) , [C10](#)
- Asking for R3 on a one-node site fails with `10005` . [C9](#)
- So: keep clients off the arbiter's client port, or fence it with placement.** [C11](#)

ONE NUMBER WORTH REMEMBERING

After the majority was gone, `/jsz` went on naming a meta leader for **tens of seconds**. [F7a](#)

And a new leader is **named before it will take a change** — ask at once and a healthy supercluster refuses. [F6a](#)

Late in one direction, early in the other. Neither edge survives a single reading.

A WHOLE REGION DIES — AND THE ANSWER FLIPS

- ZA dies. **T3 keeps 4 of 7, T4 keeps 6 of 9** — still a majority, so the survivors elect a leader and AU creates **new** streams. [C5](#) , [C6](#) , [F22](#) , [F23](#)
- The leader's **address** never mattered. Only the count does.
- ZA's data is still gone**, even with the vote alive. [F28](#)
- On slide 06 the same loss leaves 3 of 6 and freezes both. **That is what the third site buys.** [A10](#)

CHOOSE AN ARBITER WHEN

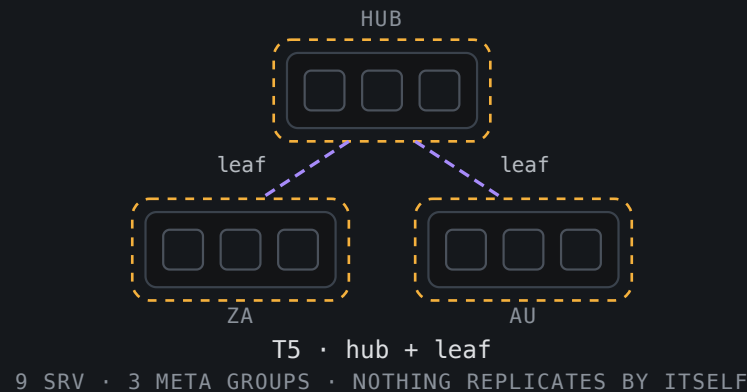
Both regions must accept JetStream **changes** while the other is dark, and you can run a third site. Take **T4**, not T3 — T3 survives the region loss with nothing left over, and a single extra node failure then freezes everything.

10

T5 · 9 SERVERS · 26 CHECKS — ALL PASSED

Independence you can have, at the price of copying by hand

With this lab's leaf link (`$SYS` not carried, a domain per side) each region keeps its own JetStream and vote.



T5 — HUB AND LEAF · THREE SEPARATE META GROUPS

- › **Three meta groups**, sizes 3 / 3 / 3. Each has its own quorum of two. [D1](#) , [D2](#)
- › **The whole hub can vanish.** Both regions keep a leader, both create new 3-replica streams, and AU keeps storing its own publishes. [D10](#) – [D14](#)
- › Both regions may own a stream called `ODOMETER` with no `10058` , because there is no shared JetStream namespace. [D3](#) , [D4](#)

T5 — WHAT IT COSTS

- › **Double capture is real here.** ONE publish in ZA was stored in ZA *and* in AU — 1 / 1 — and two same-named shadow streams lived at once. [D5](#) , [D6](#)
- › **That 1 / 1 is not the gateway's 1 / 1** — there it is one stored copy read twice. An account per region makes it 1 / 0; next slide. [D5a](#) , [D18](#) , [H6](#)
- › **Nothing replicates by itself.** Every cross-region copy is hand-written, and the panel below is why that is harder than it sounds. [D15](#)

SO HOW DOES DATA GET FROM ONE LEAF TO THE OTHER? YOU WRITE IT BY HAND.

- › A leaf link carries **messages**, not the catalogue. A name-only lookup stays local. [D1](#)
- › What works is a **mirror naming the remote API**: `external.api $JS.au.API` → 2 copied. [D8](#)
- › **Leave that line out and you get silence.** 0 messages, no error, no warning, stream healthy. [D7](#) , [D7a](#)
- › **Worse:** a name-only mirror silently copies the *local* stream of that name. [D9](#) , [D9a](#)

Rule: always write `external.api` . A green mirror is not evidence that anything was copied.

CHOOSE A LEAF LINK WHEN

Regional independence matters more than one JetStream namespace — and budget for the copiers. Share deliberately: name the remote API, or use the account wall.

11

T5 / H · 9 SERVERS · 15 CHECKS — ALL PASSED · MEASURED 2026-09-23

The fix for double capture, and the bill it hands you

The previous slide named double capture as T5's price, and per-region accounts as the way out. That is now measured, on the same nine servers. It removes ZA / AU cross-capture. It does not stop the hub capturing each region.

WHAT THE ACCOUNT WALL BUYS

- › **Cross-region double capture is gone.** One publish in `LB_ZA` → 1 in ZA, 0 in AU. On the shared account the same send was 1 / 1. `H6` , `D18` , `D18a`
- › **Still three meta groups**, 3 / 3 / 3, quorum two each. The account changes nothing about the vote. `H1` , `H2`
- › **The hub is a reader, not a dependency.** Kill all three hub servers and a region keeps its leader, creates new 3-replica streams, and stores what it accepts. `H12` – `H16` , `H14a`
- › **A whole region can go dark** and the other carries on, then both come back to 3 / 3 / 3. Measured on T5's shared account (`D24` – `D27`); inferred for H.

WHAT IT COSTS

- › **The leaf link binds ONE account.** In D the regional accounts have no leaf binding, so there they are **six separate islands**, one per site. `D22a`
- › **H gives each regional account its own remote.** The hub then captures and mirrors each region — it removes ZA / AU cross-capture, not capture at the hub. `H8` , `H11`
- › **Capture is not replication.** The hub captures only while the link is up (`H8a`). A mirror copies what the source already holds (`D8`). Catch-up after an outage was not measured.
- › **A shared hub is not a shared account.** Both regions reach the same three hub servers and still cannot see each other. `H6a` , `H9` , `H9a`
- › **The mirror trap is unchanged.** Name-only → 0 copied; with `external.api` → 2. The account was never the problem — here `external.api` names a **domain** API prefix. It can also name an API imported from another account; that was not configured here. `H10` , `H11` , `H11a`

THE SAME TWO NUMBERS, THREE DIFFERENT MEANINGS — READ THEM WITH THE SHAPE IN HAND

- › **Gateway, shared account — 1 / 1.** ONE stored copy, read from two sides. `A26` , `C13` , `F13`
- › **Leaf link, shared account — 1 / 1.** TWO stored copies of one send, with their own sequence numbers. `D5` , `D5a`
- › **Leaf link, account per region — 1 / 0.** One copy, and the other side never saw the subject. `H6`
- › **So a passing check is not a finding.** The same measurement means the opposite thing on the shape above it.

THE DECISION

If ZA / AU double capture is what frightens you, an account per region on a leaf link removes it. The hub still captures each region (`H8`). Accept in exchange that a name-only request from the hub resolves in the hub's own domain. To read a region's stream it must name that domain's API, as the `H11` mirror does — and every copy is still hand-written. `H7a` , `H8a` , `H15`

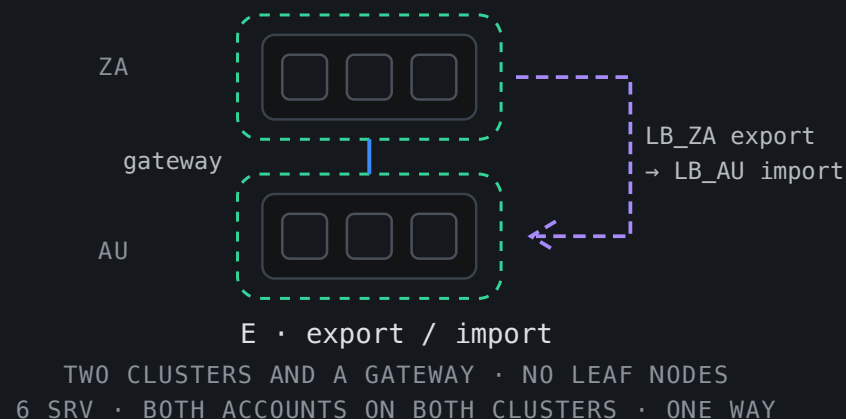
demo 03 · H1–H16 · D16–D27

12

E · 6 SERVERS · 17 CHECKS — ALL PASSED · THE ONLY SHARING IN THIS DEMO THAT IS EXPLICIT

The one explicit way to copy across a wall

E is **not** T5: two clusters, a gateway, no leaf nodes. It asks: if two **accounts** must share one subject, how do you do it on purpose, in one declared direction?



WHAT IT DOES

- › ZA publishes once. The owning account stored it, and the importing account **LB_AU** stored it too, across the gateway. [E1](#) , [E2](#)
- › **It arrives renamed.** **LB_AU** 's raw-subject stream took **none** of it — the importing side prefixes the subject. [E3](#)
- › So imported traffic sits on subjects of its own, apart from local traffic. [E3a](#) That is **not proof of origin**: a local user can publish onto the prefix too, unless permissions reserve it. The rig set none.
- › When **LB_AU** publishes the same subject itself, ZA's stream is **unchanged**. Traffic only moves the way you declared. [E4](#) , [E5](#)

THE ACCOUNT WALL STILL HOLDS

- › **LB_AU** asking for **stream info** on ZA's **ODOMETER** fails. [E6](#)
- › And **LB_AU** may create its **own** stream called **ODOMETER** with **no 10058**. [E7](#)
- › They are two real, separate streams, in **za** and **au**. [E8](#)
- › Compare slide 06: one account over a gateway is **one** JetStream namespace, and that name collides.
- › **A name-only mirror stays home.** **LB_AU** asked to mirror **ODOMETER** by name. It resolved to its own stream: 0 copied, ZA held 1. [E17](#) , [E18](#)
- › **A configured cross-account mirror is documented, not tested here.** Export the JetStream API, import it, and set **external.api**. See [NATS docs: cross-account subjects](#).

[SHARE THIS WAY](#)

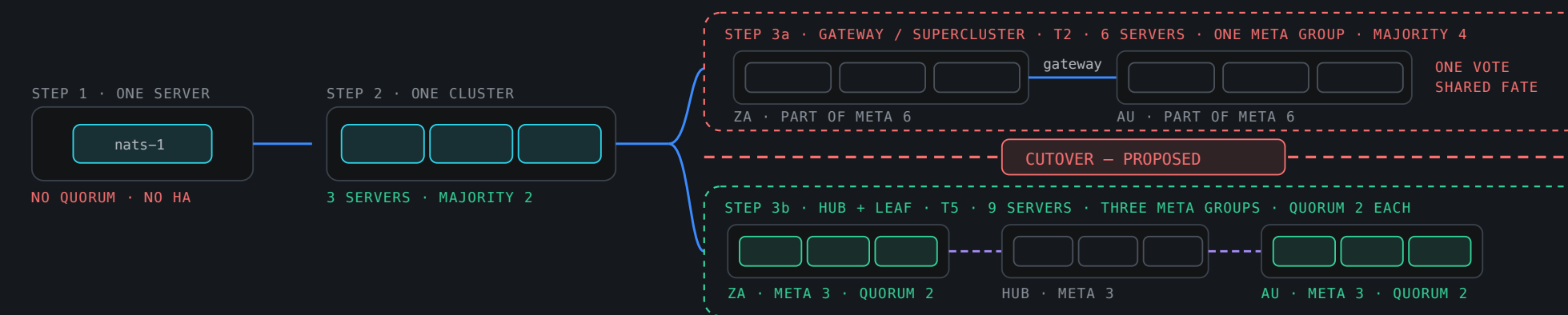
Explicit, one direction, and **renamed on arrival** — the only explicit sharing measured here. [E9](#) The prefix separates namespaces; it does not prove origin — reserve it with publish permissions. Where two tenants or regions must see one event, put an account wall between them and export across it.

13

ONE SERVER → ONE CLUSTER → TWO REGIONS · AND THE ONE STEP THAT IS NOT A STEP

Growing is cheap until the last move

Other slides show a **shape**; this one shows the **order**. Three moves are small. The last is not. T6 measured a gateway and a leaf link side by side — coexistence, not conversion (G24). One in-place conversion was measured and failed (slide 14). Others are untested.



Blue arrows and the cutover are **design, not measurement**. Measured: G24, a leaf link beside a gateway changes nothing it does; and one T4 ↔ T5 switch, which failed (slide 14).

STEPS 1 TO 3 — ORDINARY

- One server is a starting point, not a design. It has no quorum and no HA.
- Three servers in one cluster is the first real shape: majority 2, one node may die.
- Going from one server to a cluster, **every** member needs at least one route — the seed included. A routeless seed with JetStream on refuses to start.
- Step 3 is where the whole decision lives — and it is a fork, not a line.

GATEWAY AND LEAF AT ONCE — MEASURED

- T6 put a gateway **and** a hub leaf link on the same servers. Result: **two** meta groups, not three. G1, G2
- The gateway still fuses both regions into one group of 6, and both name the same meta leader. G3, G15
- 10058 comes back, and T5's double capture collapses to **one** stored copy read through two handles. G4, G7, G7a
- A dark region still freezes every create with 10008, and a healthy hub cannot lend a vote. G11a, G14a

SO DECIDE ON DAY ONE

- If a region must survive alone, go 1 → 3 → hub and leaf. Do not stop at a gateway on the way.
- If one JetStream namespace is the point, the gateway is the simpler shape — then buy the vote back with a **three**-node arbiter site. One node of slack, not zero.
- Changing your mind later: the one in-place procedure tested failed (slide 14). A parallel build and cutover is proposed and untested. Neither shape is a proven one-way choice.

PICK THE END SHAPE FIRST

Adding the hub link early buys nothing — the gateway decides while it exists. G24 The cheap moves are 1 → 3 → a second region; the expensive move is changing which kind of second region you chose.

demo 03 · T6 / G · G1–G24 · one switch measured, slide 14

14

D03-R11 · D03-R12 · LAB/09-SWITCH-T4-T5.SH · A PROCEDURE, NOT A SHAPE

One way to switch shapes in place, tested twice

The procedure: stop all nine servers, rewrite the configs, start them again on the same stores. It was run three ways, on 2026-09-28 and again on 2026-09-29. The rig held every time. The procedure did not. This tests **one** procedure. It says nothing about any other conversion or migration.

A · T4 → T5 · SA41 FAILED

- › **The target shape did not form.** Every site still reported nine members, and no site elected a meta leader in 289 s — after the switch and after a restart. SA7 , SA10 , SA28 , SA29
- › **The data held, on 09-29.** Every message, consumer position and KV key read back and matched. SA11 – SA20 , SA30 – SA39
- › Formed stream groups kept taking acked publishes. SA21 – SA27
- › On 09-28 the reader needed a meta leader, so the data was not read: unverified, not disproved.

B1 · T5 → T4, CLEAN NAMES · SB41 FAILED

- › **One group of nine formed** both times. SB10 , SB29
- › **09-28:** the ZA streams could not be read, took no new acks, and their directories were gone.
- › **09-29:** everything checked matched. SB11 – SB20 , SB30 – SB39
- › Its one unmet check is an **unread** placement right after the switch. It read za after the restart. Not evidence of movement or loss. SB18 , SB37
- › **Same procedure, two outcomes: not repeatable.**

B2 · T5 → T4, NAME COLLISION · SC59 FAILED

- › **Replica deletion observed.** za-2, then za-1, removed their copies of every ZA stream. za-3 kept its copy.
- › With one copy of three, those streams took no new acks. SC29 , SC33 , SC35
- › After the restart no replica answered for two of them: **inconclusive.** Files remain on za-3; recovery is untested. SC42 , SC48
- › **Aliasing:** both regional lookups resolved to the same AU stream. SC36 , SC51
- › The AU consumer moved on by one because the test probe acked through the ZA name — a test side effect. SC38 , SC56

WHAT THIS SAYS — AND WHAT IT DOES NOT

- › **The tested procedure has not shown safe, repeatable operation.** Do not use it on stores you need.
- › **Other procedures are untested** — a parallel build and cutover among them. A failed direct conversion does not make T4 or T5 a one-way choice.
- › **Why the replicas were removed is not proved.** Each removal follows a ZA server rolling its meta log back to the new AU leader's — but za-3 rolled back too and kept its copy.
- › **Provenance.** The 2026-09-29 run judged its verdicts live. The 2026-09-28 verdicts were derived from its evidence afterwards.

VERDICTS

D03-R11 T4 to T5 in place: failed (SA41 failed). D03-R12 T5 to T4 in place: failed (SB41 failed, SC59 failed). Replica deletion is not proved irrecoverable loss.

demo 03 · 83 procedure checks met, 18 not met · 3 procedure verdicts

15

WHAT THIS LETS SOMEBODY DECIDE

Pick the shape from the requirement, not the diagram

Each row is a requirement somebody will state, and the shape the measurements select for it.

IF THE REQUIREMENT IS...	...THE SHAPE IS	...AND THIS IS WHAT IT COSTS
One tenant must not see another tenant's data	An account. Not a cluster, not a domain.	Nothing. It is the one wall NATS enforces itself, and it holds across a gateway.
One name, one copy, no overlapping capture	Gateway (T2)	Shared availability. A 3/3 split freezes every account at once.
Both regions must accept a JetStream change while the other is dark, in one shared JetStream namespace	A vote outside both regions — T3 (7) or T4 (9). Without the shared namespace, T5 does this too.	A third site. T3 has zero slack afterwards; T4 has exactly one node .
A region must keep running when the centre is gone	Leaf links + a domain per region (T5)	Double capture on a shared account (per-region accounts remove ZA / AU cross-capture, not hub capture). Every cross-region copy is a hand-written mirror.
Two accounts must share one subject on purpose	Export / import	One-directional, read-only, renamed on arrival. The only explicit sharing measured.
Data must stay in-country (residency)	A regional cell , gateway-joined, under the one global operator	Placement discipline: the tenant is pinned, no stream mirrors or sources leave the cell.
Access and control must stay in-country (sovereignty)	A sovereign cell — own operator, own resolver, no gateway	A second provisioning act, a second key chain to rotate, a second resolver to run.
A depot or yard must survive losing its link	A leaf node	Not a cross-border answer by our policy. NATS lets a leaf bridge operators and authenticate locally; the rule is ours, not the server's.

WHERE TO START — AND STAY

- **One cluster of three.** It survives one loss, and it is the whole of what most of the platform needs.
- **Replicas: 3** set explicitly on every stream and KV bucket that matters.
- Accounts as the tenant wall from day one. Retrofitting that boundary later is the expensive move.

WHAT WE HAVE NOT MEASURED YET

- The **latency** cost of a cross-region read. Placement is measured; latency is not.
- **Mirror lag and bandwidth**, so a mirror's delay to a CQRS read model is unpriced. Slide 17.
- The hub as **primary store** under load, and the cost of losing it. It already holds data (**H8** , **H11**).
- Leaf reconnect with more than two regions.

THE NEXT QUESTION

Every shape here is available today and we can pick one on the evidence. What we cannot yet price is **the wire**: what a cross-border byte costs in latency and in bandwidth.

That is the next demo, and it is the one that decides whether a gateway between regions is affordable at all.

16 WHERE EVERY NUMBER CAME FROM

A figure with no provenance becomes a stale constant

Each number on these slides, the day it was measured, and what it ran on. No number was taken from documentation or from memory. Where a slide states documented behaviour, it links the NATS docs in place.

THE RUNS

SOURCE	MEASURED	ON	WHAT IT PRODUCED
Demo 03 — lab/run-all.sh	2026-09-28 13:40 SAST	nats-server 2.14.6 · Darwin 25.4.0 arm64	Report totals: 196 rig checks passed, 0 failed, 71 notes — 178 and 52 from 9 topologies, 18 and 19 from the switch run. Meta group sizes, 10008 / 10058 / 10005 refusals, mirror behaviour, all six findings.
Demo 03 — lab/09-switch-t4-t5.sh	2026-09-29 09:39 SAST · 2026-09-28	nats-server 2.14.6 · Darwin 25.4.0 arm64	83 procedure checks met, 18 not met, 3 procedure verdicts. 09-29 verdicts ran live; 09-28 verdicts derived afterwards. Slide 14.
Demo 02 — multi-region lab scripts	2026-09-09 & 2026-09-11	nats-server 2.14 in Docker, one host	Replicas 1 vs 3, the account wall across a gateway, WAN cut = change freeze only (10 → 15 messages acked), subject overlap 10065.
Demo 04 — cqrs rehydrate	2026-09-16	nats-server 2.14.3 in Docker, on a laptop	10 001 events on one vehicle, best of three runs each way: 25 ms without a snapshot, 1.1 ms with one.
ADR-054 / ADR-056	Accepted 2026-09-07 / 2026-09-18	V3 platform architecture record	The five axes, the tenant-wall rule, and the three cross-border shapes (leaf node, regional cell, sovereign cell).

RETRACTIONS — BOTH CORRECTIONS ARE ON THE RECORD

- › “25 / 25 / 50 km cross-region double capture” is withdrawn. A durable consumer was deleted on every projector run, so each run replayed the whole stream: 12.5, 25, 37.5, 50 over one message. The 50 added two regions reading one bucket. Behind a gateway nothing is captured twice — double capture is real in hub-and-leaf instead.
- › “8.2 s, about 600x” is withdrawn. An ordered consumer's `Next()` reset the consumer on every call — one consumer per event, about 99% of the time. Corrected: 25 ms vs 1.1 ms, about 23x.

HOW TO REPRODUCE ANY OF IT

- › Demo 03, all nine topologies: `demos/03-multi-cluster-and-accounts/lab/run-all.sh` — builds every shape from nothing and regenerates `REPORT.md` and `REPORT.html`. Both are generated; neither is hand-edited.
- › Demo 02, replicas and the account wall: `demos/02-multi-region/lab/01-the-wall.sh`, `02-replicas.sh`, `03-odometer.sh`.
- › Demo 04, rehydration: `cqrs seed -vehicle V1 -n 10000`, then `cqrs rehydrate -vehicle V1 -snapshot=false`; wait about 20 seconds for the asynchronous snapshotter, then `cqrs rehydrate -vehicle V1 -snapshot=true`. Full setup: `demos/04-jetstream-cqrs/README.md`.
- › A caution that applies to all three: never fake a WAN cut with `docker network disconnect` — it drops published host ports, and an early run wrongly reported that both regions had died.

17

STREAMS, CONSUMERS, KV AND THE COST OF REPLAY

A KV bucket is a stream, and a consumer lives where its stream lives

Placement is not a separate subject from clustering — it is the same subject seen from inside one cluster. And once state is rebuilt from a log, the length of that log becomes a latency number you have to manage.

THE CQRS SHAPE MEASURED IN DEMO 04



The log is the only source of truth. Delete both KV buckets, reset the projector consumers to the start, and a replay rebuilds them. Delete the stream and nothing does.

REHYDRATION — 10 001 EVENTS ON ONE VEHICLE

from seq 1
NO SNAPSHOT 25 ms

snapshot
+ THE TAIL 1.1 ms

ABOUT 23 TIMES FASTER — AT 10 001 EVENTS

Replay cost grows with the length of the log. The snapshot read is one KV get plus whatever tail is left, so its cost follows the **tail**, not the log. A tenfold longer log should cost roughly tenfold more to replay — an extrapolation, not a measurement.

PLACEMENT — THE RULES WE MEASURED

- › Placement is **per stream**. A new stream lands in the cluster of the client that creates it, unless you name one with `--cluster`. The account and the meta leader do not pin it.
- › A **KV bucket follows the same rule, because a bucket is a stream**.
- › A durable pull consumer created from AU for a ZA-held stream reported cluster **za**. AU held no part of it and paid the WAN on every pull.

Not measured. A local mirror takes the WAN off reads but adds its lag to every read model built on it. A cut-off mirror serves old data with no error. Check commands against the source, never a mirror.

TWO SETTINGS THAT FAIL SILENTLY

- › **LimitsPolicy, not InterestPolicy**, on any stream you intend to replay. InterestPolicy discards a message once every consumer has acked it, and the replay then returns nothing — with no error.
- › **The snapshot may be stale.** Rehydration must read the snapshot *and then replay the tail*. Measured mid-catch-up: 2 900 events of tail in 12 ms.

STORAGE — THE STANDING RULE

- › **A count is never shown without its bytes.** 100 000 000 events sounds reasonable right up to the moment you learn it is about **7.6 GB** — extrapolated from about 76 bytes per event.
- › R3 triples that figure, per stream, per region.
- › Seeding one million events into the bench fixture took **2.195 seconds**, measured — the log grows faster than anyone plans for.

RECOMMENDATION

Treat snapshotting as a **requirement**, not an optimisation, on any aggregate that will take more than a few thousand events. And state a retention policy on day one, in bytes.

18

FINDINGS THIS LAB ADDED · THE SIXTH ONE IS SLIDE 13

Five failures that report success

Not in the documentation. Each was found by a script that expected one answer and got another. Every one looks healthy from the outside — which is why they are here.

01 · THE MONITORING ENDPOINT LIES WHILE IT IS STALE

After the majority of a supercluster was frozen, `/jsz` kept naming a meta leader for **roughly a minute**. Read it too early and a dead supercluster looks perfectly healthy.

In that window the client does not get a clean refusal either — **it hangs and times out**. The clean `10008` only appears once the old leader has aged out.

02 · A NEW LEADER IS NAMED BEFORE IT CAN TAKE A CHANGE

After an election, `/jsz` names the new leader **seconds before** it will accept a create. Ask at once and a healthy supercluster refuses.

Neither edge of a failover can be trusted from one reading. Readiness is a real operation that succeeds, not a name in `/jsz`. Retry, and record the wait.

03 · A NAME-ONLY MIRROR CAN COPY THE WRONG STREAM

A mirror with no `external.api` does not fail when the name it wants only exists in **another domain** (this lab's leaf link) — **it just sits at zero**. Across a gateway there is one domain, and a name alone works (`A15` , `A16`). Worse: if a stream of the same name also exists locally, the mirror copies *that* one, reports healthy, and looks like it is working.

Proved by giving the two streams different message counts. No error, no warning, wrong data.

04 · WHICH REGION GOES BLIND IS A COIN TOSS

Two runs of the same configuration, minutes apart, nothing changed — and the blind side swapped.

A team that tests this shape once will write down the wrong region's name. Our check deliberately refuses to name a side.

05 · A TEST THAT DOES NOT TEST READS EXACTLY LIKE A RESULT

Not a NATS fact — a rig fact, and the most important one. An early version of the harness recorded the wrong process id, so `kill -STOP` hit nothing and the “frozen” region stayed fully alive. **The lab reported healthy behaviour and it looked like a finding.** Every freeze in these scripts now proves itself first, by checking the monitor port has really gone dark.

TWO NUMBERS WE RETRACTED

- › A cross-region “double capture” of **50 km** was a replay bug — a durable consumer was being deleted on every run.
- › A rehydration speed-up of **600x** was **99% consumer bookkeeping**, not log reading. The real figure is 23x.
- › **A lab whose numbers only ever improve is not measuring.**

OPERATIONAL CONSEQUENCE

Do not build alerting, failover automation or a runbook on a single read of `/jsz`. Every one of these failures passes a naive health check.